

Autonomous Driving Vehicle

Contents

1	Hardware	2
2	Software	6
2.1	Architecture	6
2.1.1	Embedded Platform	6
2.1.2	Brain	7
2.2	Embedded Platform	8
2.2.1	Peripheral	8
2.2.2	Sensor	12
2.2.3	Actuator	15
2.3	Brain	15
2.3.1	Module Communication	15
2.4	IO	18
2.4.1	Perception	18
2.4.2	Localization	19
2.4.3	Planning	19
2.4.4	Control	19
2.5	Monitor	19
2.6	Simulation	19
3	Source & Description	19

1 Hardware

Vehicle Platform

STM32F401 NUCLEO

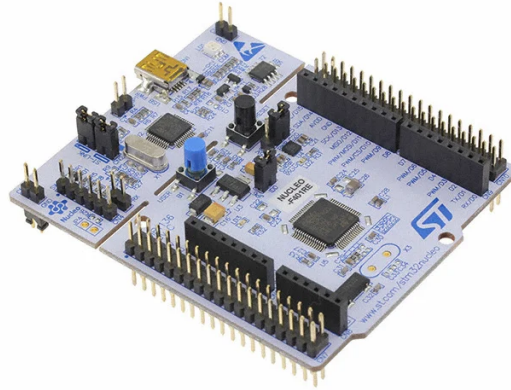


Figure 1: STM32F401 NUCLEO board

Property	Value
MCU	STM32F401RE (ARM Cortex-M4)
Clock	Up to 84 MHz
Flash	512 KB
RAM	96 KB
Interface	UART, I2C, SPI, GPIO, ADC, PWM
Role	Low-level control: motor, servo, sensor reading

Table 1: STM32F401 NUCLEO specifications

Raspberry Pi 5



Figure 2: Raspberry Pi 5

Property	Value
CPU	Broadcom BCM2712, quad-core Cortex-A76 @ 2.4 GHz
RAM	4 GB / 8 GB LPDDR4X
OS	Ubuntu 24.04 (64-bit)
Interface	UART, USB, CSI camera, Ethernet, Wi-Fi
Role	High-level brain: perception, planning, control

Table 2: Raspberry Pi 5 specifications

Encoder: AS5600

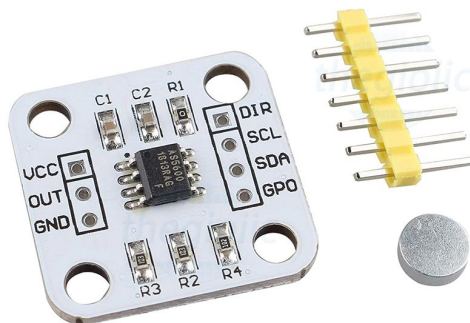


Figure 3: AS5600 magnetic rotary encoder module

Property	Value
Type	Contactless magnetic rotary encoder
Resolution	12-bit (4096 positions/revolution)
Interface	I2C (address 0x36)
Supply voltage	3.3 V – 5 V
Output	Absolute angle 0°–360°
Role	Measures wheel rotation for speed estimation

Table 3: AS5600 encoder specifications

IMU: BNO055

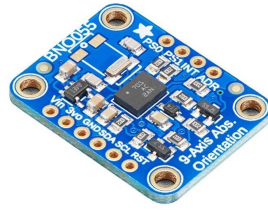


Figure 4: BNO055 9-axis IMU module

Property	Value
Axes	9-axis (accelerometer + gyroscope + magnetometer)
Interface	I2C (address 0x28 / 0x29)
Output	Euler angles: yaw, pitch, roll (degrees)
Supply voltage	3.3 V
Fusion mode	NDOF (on-chip sensor fusion)
Role	Provides heading angle (yaw) for state estimation

Table 4: BNO055 IMU specifications

Actuator: Steering Servo



Figure 5: 180° steering servo

Property	Value
Type	PWM position servo
Range	0°–180°
Control	PWM 50 Hz, pulse width 1–2 ms
Supply voltage	5 V
Role	Controls front wheel steering angle

Table 5: Steering servo specifications

Simulator

- Encoder: AS5600
- Motor
- 3D Model design

2 Software

2.1 Architecture

2.1.1 Embedded Platform

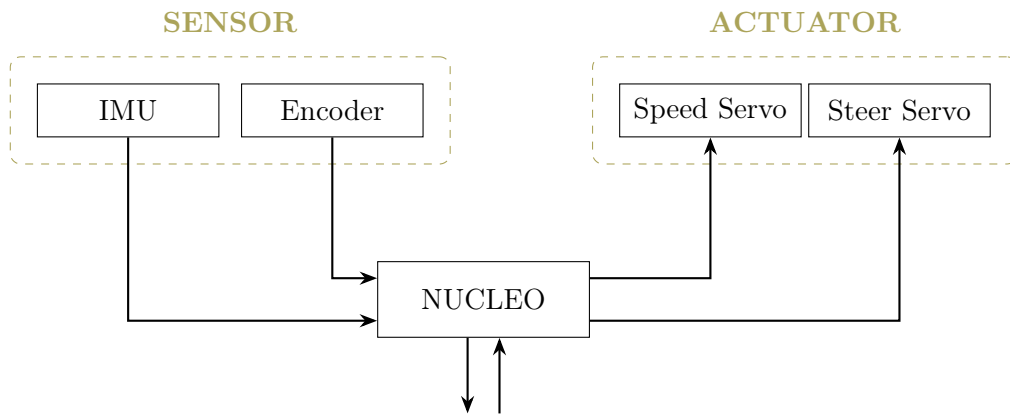


Figure 6: Embedded Platform: NUCLEO reads sensors via I2C, drives actuators via PWM, communicates with Raspberry Pi 5 via UART

2.1.2 Brain

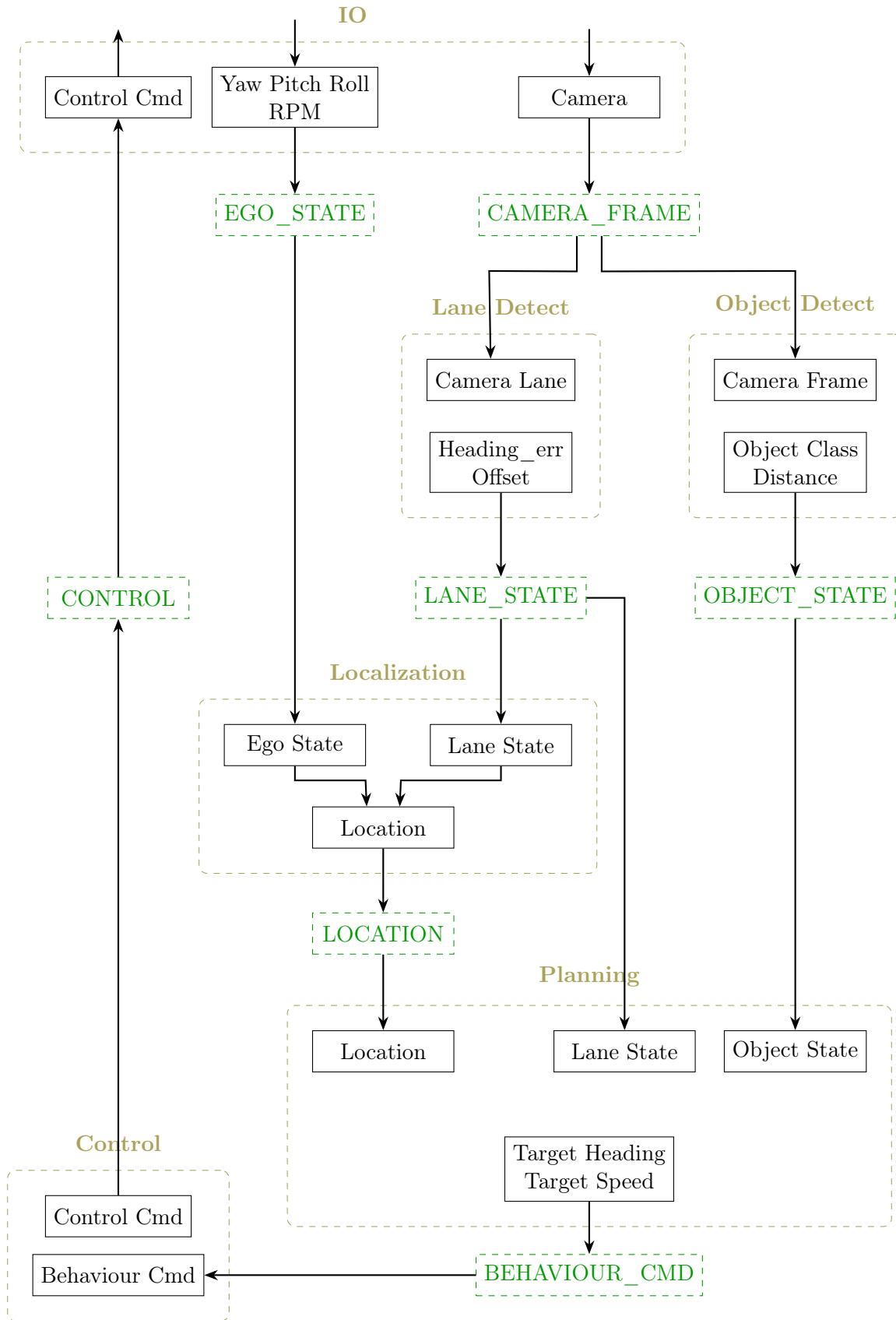


Figure 7: Brain architecture: IO layer

2.2 Embedded Platform

2.2.1 Peripheral

I2C (Inter-Integrated Circuit)

I2C is a two-wire synchronous serial protocol using a shared clock line (SCL) and a bidirectional data line (SDA). Communication is initiated by a START condition (SDA falls while SCL is high), followed by a 7-bit device address, a read/write bit, an ACK from the slave, data bytes, and a STOP condition (SDA rises while SCL is high). In this project, I2C1 (PB8=SCL, PB9=SDA) communicates with BNO055 (0x28) and AS5600 (0x36) at 100 kHz.

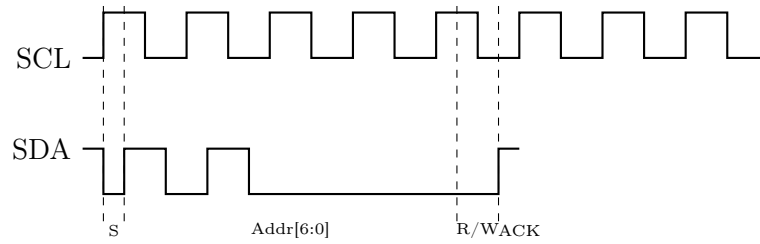


Figure 8: I2C transaction: START, 7-bit address (0x28), R/W bit, ACK

Function: I2C1_Master_Init()

Configures GPIO PB8/PB9 as AF4 (open-drain, pull-up), enables clocks, then sets I2C1 timing for 100 kHz with PCLK1 = 16 MHz.

Register	Field	Value	Effect
RCC->AHB1ENR	GPIOBEN	= (1<<1)	Enable GPIOB clock
RCC->APB1ENR	I2C1EN	= (1<<21)	Enable I2C1 clock
GPIOB->MODER	PB8,PB9	0b10 each	Alternate function mode
GPIOB->OTYPER	PB8,PB9	1 each	Open-drain output
GPIOB->OSPEEDR	PB8,PB9	0b11 each	High speed
GPIOB->PUPDR	PB8,PB9	0b01 each	Pull-up
GPIOB->AFR[1]	PB8,PB9	0x4 each	AF4 = I2C1
I2C1->CR1	PE	0 (clear)	Disable I2C before config
I2C1->CR2	FREQ	16	PCLK1 = 16 MHz
I2C1->CCR	CCR	80	100 kHz: $T_{high} = T_{low} = \frac{80}{16\text{MHz}} = 5 \mu s$
I2C1->TRISE	TRISE	17	Max rise time: $\frac{1000 ns}{62.5 ns} + 1 = 17$
I2C1->CR1	ACK	= (1<<10)	Enable auto-ACK
I2C1->CR1	PE	= (1<<0)	Enable I2C peripheral

Table 6: I2C1_Master_Init() register configuration

Function: I2C1_WriteReg(dev, reg, data)

Sends START → device address (write) → register address → data byte → STOP.

Register	Action	Value / Condition	Purpose
I2C1->CR1	Write	= START (1<<8)	Generate START condition
I2C1->SR1	Poll	wait SB==1	START sent
I2C1->DR	Write	dev<<1	Send address + Write bit
I2C1->SR1/SR2	Read	wait ADDR==1, clear by reading SR2	Address ACKed
I2C1->SR1	Poll	wait TXE==1	Tx buffer empty
I2C1->DR	Write	reg	Send register address
I2C1->SR1	Poll	wait TXE==1	Tx buffer empty
I2C1->DR	Write	data	Send data byte
I2C1->SR1	Poll	wait BTF==1	Byte transfer finished
I2C1->CR1	Write	= STOP (1<<9)	Generate STOP condition

Table 7: I2C1_WriteReg() register sequence

Function: I2C1_ReadMulti(dev, reg, buf, len)

Sends register address (write phase), then Repeated START, reads `len` bytes (NACK + STOP on last byte).

Register	Action	Value / Condition	Purpose
I2C1->CR1	Write	= START	START condition
I2C1->DR	Write	dev<<1	Address + Write
I2C1->DR	Write	reg	Register pointer
I2C1->SR1	Poll	wait BTF==1	Write phase done
I2C1->CR1	Write	= START	Repeated START
I2C1->DR	Write	(dev<<1) 1	Address + Read bit
I2C1->CR1	Write (last byte)	&= ~ACK	Disable ACK before last byte
I2C1->CR1	Write (last byte)	= STOP	STOP after last byte
I2C1->SR1	Poll each byte	wait RXNE==1	Data register not empty
I2C1->DR	Read	buf[i]	Read received byte

Table 8: I2C1_ReadMulti() register sequence

PWM (Pulse Width Modulation)

PWM generates a periodic digital signal with fixed frequency and variable duty cycle. The STM32 timer compares the counter value against a capture/compare register (CCR); output is high when $CNT < CCR$ and low otherwise. PWM is used to control the servo steering angle and to send throttle commands to the ESC. Typical servo period is 20 ms (50 Hz); pulse width 1–2 ms maps to 0°–180°.

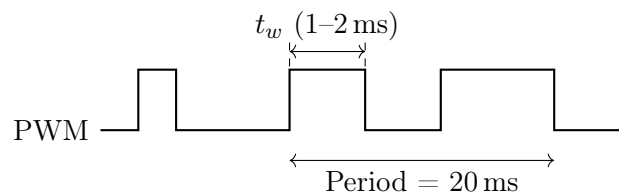


Figure 9: PWM signal: varying pulse width, fixed 50 Hz period

Function: PWM_Init()

Configures PA0 (TIM2_CH1, speed servo) and PA1 (TIM2_CH2, steering servo) as AF1 PWM outputs at 50 Hz with PCLK1 = 16 MHz.

Register	Field	Value	Effect
RCC->AHB1ENR	GPIOAEN	= (1<<0)	Enable GPIOA clock
RCC->APB1ENR	TIM2EN	= (1<<0)	Enable TIM2 clock
GPIOA->MODER	PA0, PA1	0b10 each	Alternate function mode
GPIOA->AFR[0]	PA0, PA1	0x1 each	AF1 = TIM2_CH1/CH2
TIM2->PSC	PSC	15 (16-1)	Tick = 16 MHz/16 = 1 MHz
TIM2->ARR	ARR	19999 (20000-1)	Period = 20 ms → 50 Hz
TIM2->CCR1	CCR1	1500	Speed servo: neutral (1.5 ms)
TIM2->CCR2	CCR2	1500	Steering servo: center (1.5 ms)
TIM2->CCMR1	OC1M	0b110 (bits [6:4])	PWM mode 1 on CH1
TIM2->CCMR1	OC1PE	1 (bit 3)	Preload enable CH1
TIM2->CCMR1	OC2M	0b110 (bits [14:12])	PWM mode 1 on CH2
TIM2->CCMR1	OC2PE	1 (bit 11)	Preload enable CH2
TIM2->CCER	CC1E	1 (bit 0)	Enable CH1 output
TIM2->CCER	CC2E	1 (bit 4)	Enable CH2 output
TIM2->EGR	UG	1 (bit 0)	Force reload PSC/ARR
TIM2->CR1	CEN	= (1<<0)	Start counter

Table 9: PWM_Init() register configuration

Functions: Servo_Speed_Set(us), Servo_Steering_Set(us)

Function	Register	Value	Range
Servo_Speed_Set(us)	TIM2->CCR1	us	1000–2000 μ s
Servo_Steering_Set(us)	TIM2->CCR2	us	1100–1900 μ s
Servo_Speed_Set_RPM(rpm)	TIM2->CCR1	1500 - (int)rpm	rpm: -500..+500
Servo_Steering_Set_Deg(deg)	TIM2->CCR2	1500 + (int)(deg*10)	deg: -40..+40°

Table 10: Servo control functions and their CCR values

UART (Universal Asynchronous Receiver/Transmitter)

UART is an asynchronous serial protocol with no shared clock. A frame consists of: 1 start bit (low), 8 data bits (LSB first), optional parity, and 1–2 stop bits (high). Both ends must agree on the same baud rate. This project uses 115200 baud, 8N1 (no parity, 1 stop bit). The STM32 transmits sensor data to the Raspberry Pi 5 over UART in ASCII frames: @IMU:yaw,pitch,roll;;\r\n and @RPM:rpm;;\r\n.

Function: UART1_init() — PA9 TX, PA10 RX, AF7, 115200 baud 8N1, PCLK2 = 16 MHz.

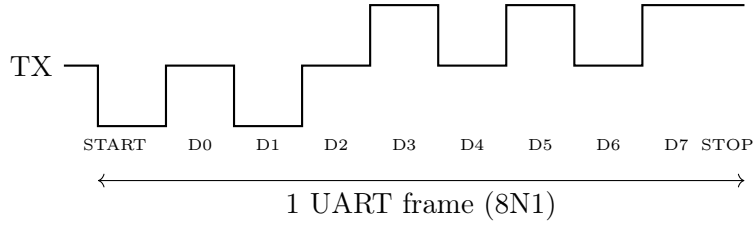


Figure 10: UART frame: start bit, 8 data bits (LSB first), stop bit

Register	Field	Value	Effect
RCC->AHB1ENR	GPIOAEN	= (1<<0)	Enable GPIOA clock
RCC->APB2ENR	USART1EN	= (1<<4)	Enable USART1 clock
GPIOA->MODER	PA9, PA10	0b10 each	Alternate function mode
GPIOA->AFR[1]	PA9, PA10	0x7 each	AF7 = USART1
USART1->BRR	DIV_Mantissa	8<<4	115200 baud: USARTDIV = 8.6875
	DIV_Fraction	11	
USART1->CR1	OVER8	0 (bit 15)	Oversampling by 16
USART1->CR1	M	0 (bit 12)	8-bit word length
USART1->CR1	PCE	0 (bit 10)	No parity
USART1->CR1	RE	1 (bit 2)	Receiver enable
USART1->CR1	TE	1 (bit 3)	Transmitter enable
USART1->CR1	UE	1 (bit 13)	USART enable

Table 11: UART1_init() register configuration

Function: UART2_init() — PA2 TX, PA3 RX, AF7, 115200 baud, with RXNE interrupt (NVIC IRQ 38).

Register	Field	Value	Effect
RCC->APB1ENR	USART2EN	= (1<<17)	Enable USART2 clock
GPIOA->MODER	PA2, PA3	0b10 each	Alternate function mode
GPIOA->AFR[0]	PA2, PA3	0x7 each	AF7 = USART2
USART2->BRR		(8<<4) 11	115200 baud @16 MHz
USART2->CR1	RXNEIE	1 (bit 5)	Enable RXNE interrupt
USART2->CR1	RE, TE, UE	1 each	Enable RX, TX, USART
NVIC->ISER[1]	IRQ38	1<<(38-32)	Enable USART2 IRQ in NVIC

Table 12: UART2_init() register configuration

Functions: UART1_Transmit(data), UART2_Transmit(data)

Register	Action	Condition / Value	Purpose
USARTx->SR	Poll	wait TXE==1 (bit 7)	TX buffer empty
USARTx->DR	Write	data	Load byte to transmit
USARTx->SR	Poll	wait TC==1 (bit 6)	Transmission complete

Table 13: UART transmit sequence

ISR: USART2_IRQHandler() — Triggered on each received byte, reads USART2->DR to clear RXNE, passes byte to *UART_Parse_Byte()* which accumulates into a command buffer and calls *UART_Parse_Command()* on ; ; delimiter.

RCC (Reset and Clock Control)

The RCC peripheral on the STM32F401 manages system clock configuration and peripheral clock gating. The internal HSI oscillator (16 MHz) or external HSE crystal can be fed into the PLL to generate the system clock (SYSCLK). On the STM32F401 NUCLEO, SYSCLK is configured to 84 MHz. Peripheral buses APB1 (max 42 MHz) and APB2 (max 84 MHz) are derived via prescalers. Each peripheral must have its bus clock explicitly enabled via *RCC_APBxENR* before use.

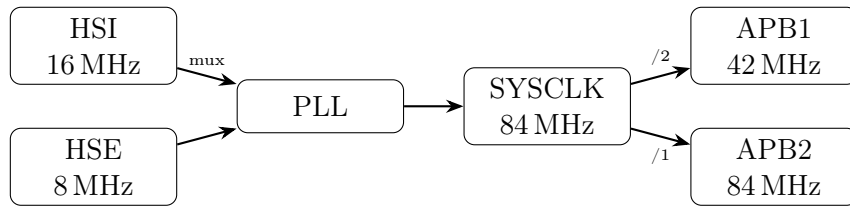


Figure 11: STM32F401 clock tree (simplified): HSI/HSE → PLL → SYSCLK → APB buses

Timer

STM32 general-purpose timers (TIM2–TIM5) use an up-counter that increments each tick. The tick frequency is $\frac{f_{APB} \times 2}{PSC + 1}$; the counter resets at *ARR* (Auto-Reload Register), producing an overflow period of $\frac{ARR + 1}{f_{tick}}$. In this project:

- **TIM1/TIM4** — PWM output for servo and ESC ($f = 50$ Hz, $ARR = 19999$, $PSC = 83$)
- **TIM2** — encoder interface mode, counts quadrature edges from AS5600 (via GPIO simulation)
- **TIM3** — periodic interrupt for UART transmit scheduling (1 kHz)

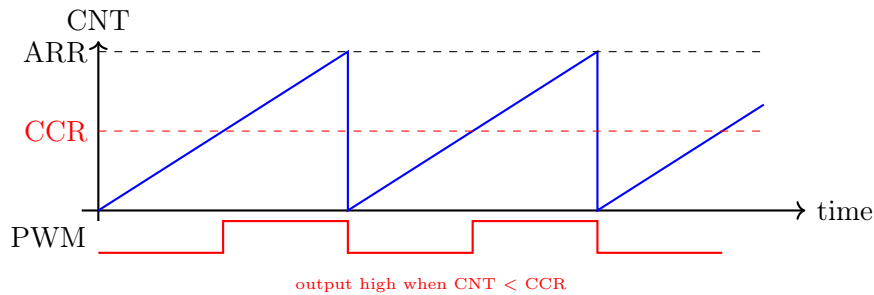


Figure 12: Timer up-count mode: counter ramps to *ARR*, resets; PWM output high while $CNT < CCR$

2.2.2 Sensor

IMU: BNO055

Communication: I2C, address 0x29 (ADDR pin high). The MCU is master; BNO055 is slave. Bus speed 100 kHz matching `I2C1_Master_Init()`.

Key registers used in implementation:

Macro	Address	Description
BNO055_CHIP_ID	0x00	Chip ID — expected value 0xA0
BNO055_PAGE_ID	0x07	Page select — write 0x00 to use page 0
BNO055_OPR_MODE	0x3D	Operation mode: 0x00=CONFIG, 0x0C=NDOF
BNO055_PWR_MODE	0x3E	Power mode: 0x00=Normal, 0x01=LowPower, 0x02=Suspend
BNO055_UNIT_SEL	0x3B	Unit select: bit0=0 → degrees output
BNO055_EUL_X_LSB	0x1A	Euler heading (yaw) LSB — 1 LSB = 1/16°
BNO055_EUL_X_MSB	0x1B	Euler heading (yaw) MSB
BNO055_EUL_Y_LSB	0x1C	Euler pitch LSB
BNO055_EUL_Y_MSB	0x1D	Euler pitch MSB
BNO055_EUL_Z_LSB	0x1E	Euler roll LSB
BNO055_EUL_Z_MSB	0x1F	Euler roll MSB
BNO055_CALIB_STAT	0x35	Calibration status — bits [7:6]=sys, [5:4]=gyro, [3:2]=accel, [1:0]=mag

Table 14: BNO055 registers used in implementation

Function: BNO055_Init()

Step	I2C Write reg	Value	Purpose
1	OPR_MODE (0x3D)	0x00	Enter CONFIG mode before changing settings
2	PAGE_ID (0x07)	0x00	Select page 0 (sensor/output registers)
3	PWR_MODE (0x3E)	0x00	Set Normal power mode
4	OPR_MODE (0x3D)	0x0C	Enter NDOF fusion mode (9-DOF)

Table 15: BNO055_Init() I2C write sequence

Function: BNO055_ReadEuler_raw(x, y, z)

Burst-reads 6 bytes starting from EUL_X_LSB (0x1A). Each axis is a signed 16-bit little-endian value; 1 LSB = 1/16°.

I2C operation	Reg	Bytes	Decode
Read 6 bytes from	0x1A	b[0..5]	
Heading (yaw) X	0x1A-0x1B	b[1]«8 b[0]	int16_t, divide by 16 → degrees
Pitch Y	0x1C-0x1D	b[3]«8 b[2]	int16_t, divide by 16 → degrees
Roll Z	0x1E-0x1F	b[5]«8 b[4]	int16_t, divide by 16 → degrees

Table 16: BNO055_ReadEuler_raw() data decode

Function: BNO055_ReadCalibStatus()

Reads 1 byte from CALIB_STAT (0x35). Each sensor pair of bits = 3 means fully calibrated.

Bits	Sensor	Value 3 = fully calibrated
[7:6]	System	Overall fusion calibration
[5:4]	Gyro	Gyroscope
[3:2]	Accel	Accelerometer
[1:0]	Mag	Magnetometer

Table 17: CALIB_STAT bit fields

Function: IMU_HealthCheck(ex, ey, ez)

Called after each BN0055_ReadEuler_raw(). If all three axes are zero for 5 consecutive reads, the IMU is flagged as faulty and the watchdog (Feed_WD()) stops being fed — triggering a system reset.

Encoder: AS5600

Communication: I2C, fixed address 0x36. No init sequence required — the device is ready immediately after power-on. Only reading is performed at runtime.

Key registers used in implementation:

Macro	Address	Description
AS5600_STATUS	0x0B	Magnet status: bit5=MH (too strong), bit4=ML (too weak), bit3=M
AS5600_ANGLE_MSB	0x0E	Processed angle high byte [11:8]
AS5600_ANGLE_LSB	0x0F	Processed angle low byte [7:0]

Table 18: AS5600 registers used in implementation

Function: AS5600_ReadStatus()

I2C operation	Reg	Bytes	Check
Read 1 byte	0x0B	v	bit3 (MD) must be 1 for valid readings

Table 19: AS5600_ReadStatus() operation

Function: AS5600_ReadAngleRaw()

I2C operation	Reg	Bytes	Decode
Read 2 bytes	0x0E	buf[0..1]	raw = (buf[0]«8 buf[1]) & 0xFFFF

Table 20: AS5600_ReadAngleRaw() — returns 12-bit value 0–4095

Function: AS5600_ReadAngleDeg()

Calls AS5600_ReadAngleRaw(), converts with:

$$\text{degrees} = \frac{\text{raw} \times 360}{4096}$$

Function: AS5600_ReadAngularSpeedDeg(dt_sec)

Computes angular velocity in $^{\circ}/s$ from two consecutive raw readings. Handles $0^{\circ}/360^{\circ}$ wrap-around by treating a delta > 2048 or < -2048 as a wrap.

Step	Description
1	Read curr_raw via AS5600_ReadAngleRaw()
2	delta = (int16_t)(curr_raw - prev_raw)
3	If delta > 2048 : delta -= 4096 (wrapped CW)
4	If delta < -2048 : delta += 4096 (wrapped CCW)
5	delta_deg = delta * 360.0f / 4096.0f
6	Return delta_deg / dt_sec ($^{\circ}/s$)

Table 21: AS5600_ReadAngularSpeedDeg() algorithm

2.2.3 Actuator

Steering: Angle 180 Servo

- ...

Motivation: Brushless Motor & ESC

- ...

2.3 Brain

2.3.1 Module Communication

Kernel

...

Inter Process Communication

Inter-Process Communication (IPC) refers to mechanisms that allow independent OS processes to exchange data. On Linux, the kernel provides several IPC primitives, each with different trade-offs in latency, throughput, ordering guarantees, and persistence.

Mechanism	Characteristics	Use case	Used in project
POSIX Message Queue	Kernel-buffered, ordered FIFO queue; fixed max depth (<code>msg_max</code>); supports blocking/non-blocking send/recv	Low-rate, queued messages where every message must be delivered	Yes — MQ transport
Shared Memory (SHM)	Fastest IPC: processes map the same physical page; no kernel copy; requires explicit synchronization	High-rate data where only the latest value matters (sensor frames, state)	Yes — SHM latest-wins
Unix Domain Socket	Stream or datagram socket over a file path; reliable, ordered; higher latency than SHM	Bidirectional communication between local processes	Not used
TCP Socket	Network socket; works across machines; adds TCP overhead	Remote monitoring, cross-host communication	Not used

Table 22: Linux IPC mechanism comparison

This project implements two transports inside `libipc`:

- **POSIX MQ** (`mq_transport.c`): wraps `mq_open/mq_send/mq_receive`. Queue depth is clamped to the system limit (`/proc/sys/fs/mqueue/msg_max`, default 10). Used for topics where every message must be consumed (e.g. `CONTROL_CMD`).
- **Shared Memory** (`shm.c`): publisher creates a named SHM object via `shm_open` + `ftruncate`, writes a fixed-size slot protected by a `pthread_rwlock`. Subscriber opens (does *not* create) the same object and reads the latest value. Used for high-rate sensor data where stale frames are discarded (e.g. `CAMERA_FRAME`, `EGO_STATE`).

Pub/Sub Model

The project adopts a publish/subscribe pattern on top of the two transports. Each logical data stream is identified by a *topic*; nodes are decoupled — a publisher does not know who is listening, and a subscriber does not know the publisher’s identity.

- **Node** — an independent OS process that owns one or more publishers or subscribers. Each node links against `liblibipc` and calls the C API directly.
- **Topic** — a named channel identified by a `TopicId` enum value (e.g. `CAMERA_FRAME=1`, `SERIAL=2`, `EGO_STATE=3`, `LANE_STATE=4`, `CONTROL_CMD=5`). Each topic has a fixed payload type, transport kind (MQ or SHM), and drop policy, declared in `topic.c`.

- **Publish** — a node calls `ipc_publish()` with a topic ID and a pointer to a POD struct. The bus layer routes the write to the correct transport. The publisher must start before any subscriber.
- **Subscribe** — a node calls `ipc_subscribe()` then `ipc_recv()` (blocking or timed). For SHM topics the subscriber receives the latest written value; for MQ topics it dequeues in FIFO order.

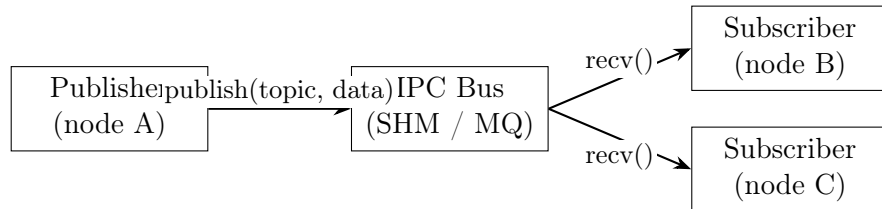


Figure 13: Pub/Sub model: publisher writes to the bus; multiple subscribers receive independently

Drop Policy

Drop policy defines what happens when a new message arrives and the buffer is full, or when a subscriber reads slower than the publisher writes.

Policy	Behavior	Transport
NEW	Drop the <i>incoming</i> message if buffer full; keep existing messages	MQ
OLD	Drop the <i>oldest</i> message in buffer to make room for the new one	MQ
NEVER	Latest write always overwrites the single slot; subscriber always gets newest value	SHM

Table 23: Drop policies in `libipc`

In this project all SHM topics use **NEVER** (latest-wins); MQ topics use **NEW** (block new if full).

C API / C++ Binding

The C API (`libipc/include/ipc/bus.h`) exposes four functions:

- `ipc_publish_open(TopicId)` — allocate publisher handle, create SHM/MQ object.
- `ipc_publish(handle, const void *data)` — write one message.
- `ipc_publish_close(handle)` — release handle, unlink SHM/MQ.
- `ipc_subscribe_open(TopicId) / ipc_recv(handle, void *out, int timeout_ms) / ipc_subscribe_close`.

The C++ binding (`libipc/bindings/cpp/include/ipc/bus.hpp`) wraps these into RAII templates: `ipc::Publisher<T>` and `ipc::Subscriber<T>`, where `T` is the message struct type. Constructors call `open`; destructors call `close` automatically.

Python Binding

The Python binding (`libipc/bindings/python/ipc.py`) uses `ctypes` to call the shared library `liblibipc.so` at runtime. Message structs are defined in `ipc_schema.py` as `ctypes.Structure` subclasses mirroring the C headers in `messages/`. Usage pattern:

1. `import ipc, ipc_schema` (after inserting the bindings path into `sys.path`).
2. `pub = ipc.Publisher(ipc_schema.TopicId.LANE_STATE)` — open publisher.
3. Fill a `ipc_schema.LaneState()` struct and call `pub.publish(msg)`.
4. `sub = ipc.Subscriber(ipc_schema.TopicId.EGO_STATE); msg = sub.recv(timeout_ms=100)`

2.4 IO

- Serial Bridge
- Web Server / Web Socket

2.4.1 Perception

Camera

- Input
- Output
- Algorithm

Lane Detection

- Input
- Output
- Algorithm

Object Detection

- Input
- Output
- Algorithm

State

- Input
- Output
- Algorithm

2.4.2 Localization

- Input
- Output
- Algorithm

2.4.3 Planning

- Input
- Output
- Algorithm

2.4.4 Control

- Input
- Output
- Algorithm

2.5 Monitor

- Web Server
- Dashboard

2.6 Simulation

- Visualization
- Direction Simulation

3 Source & Description